

[Vai al contenuto](#)

DISPENSA DELLA LEZIONE

MySQL e database relazionali

Dalle entità del mondo reale alle tabelle: campi, record, tipi di dato e vincoli.



CAPITOLO 1

Database relazionali e MySQL

NOTA

Un **database** conserva dati organizzati. Un **DBMS** è il programma che permette di crearli, proteggerli e consultarli. MySQL è un DBMS relazionale.

Un negozio, una biblioteca o una scuola devono conservare molti dati e ritrovarli con precisione. Un database li mantiene anche dopo la chiusura dell'applicazione e consente a più persone o programmi di lavorarci in modo controllato.

Nei database relazionali i dati sono organizzati in **tabelle**. Questa dispensa si concentra sulla progettazione e definizione di una singola tabella.

MySQL riceve istruzioni scritte in SQL. Qui usiamo SQL per definire la struttura di una tabella, senza affrontare l'inserimento o la ricerca dei dati.

CAPITOLO 2

Entità e attributi

NOTA

Un buon attributo contiene un solo fatto. Per esempio, è meglio avere `nome` e `cognome` separati che un unico campo `nome_completo`, se prevediamo di usarli separatamente.

Prima di aprire MySQL conviene descrivere il problema. Un'**entità** è un concetto del mondo reale che vogliamo memorizzare: un cliente, un libro, un prodotto.

Gli **attributi** sono le caratteristiche dell'entità. Per l'entità `Cliente` possiamo usare nome, cognome, email e data di nascita.

Entità: Cliente

Attributi:

- nome
- cognome
- email
- data di nascita

Evitiamo anche i dati che cambiano automaticamente nel tempo. L'età, per esempio, diventa presto sbagliata: è più utile salvare `data_nascita`.

CAPITOLO 3

Dall'entità alla tabella

NOTA

Una tabella descrive la struttura dei dati; una riga contiene i dati di un singolo elemento.

Nel database l'entità **Cliente** diventa la tabella **clienti**. I suoi attributi diventano **campi**, chiamati anche **colonne**.

Tabella: clienti

id	nome	cognome	email
1	Mario	Rossi	mario@example.com
2	Anna	Bianchi	anna@example.com

- **Campi o colonne:** **id**, **nome**, **cognome** ed **email**.
- **Tuple, record o righe:** i dati di Mario Rossi e Anna Bianchi.

I termini tuple, record e riga indicano qui la stessa idea: un insieme di valori riferiti a una sola entità concreta.

CAPITOLO 4

Tipi di dato

Ogni campo deve dichiarare il tipo di valore che può contenere. La scelta del tipo aiuta MySQL a salvare, confrontare e controllare i dati nel modo corretto.

- **INT** per numeri interi, come un identificatore o la quantità disponibile.
- **VARCHAR(n)** per testi brevi fino a **n** caratteri, come nome o email.
- **DECIMAL(p, s)** per numeri con decimali esatti, come prezzi e importi.
- **DATE** per una data, come la data di nascita.
- **BOOLEAN** per un valore vero/falso, come un cliente attivo.

prezzo	DECIMAL(8, 2)	→	19.90
data_nascita	DATE	→	2004-09-16
attivo	BOOLEAN	→	TRUE

VARCHAR(50) non significa che tutti i valori devono avere 50 caratteri: indica il limite massimo. Per un prezzo, invece, evitiamo **FLOAT** e usiamo **DECIMAL**, perché gli importi richiedono precisione.

CAPITOLO 5

Chiave primaria e vincoli

NOTA

I **vincoli** sono regole applicate da MySQL per impedire l'inserimento di dati non validi o incoerenti.

Ogni tabella deve poter distinguere senza ambiguità i suoi record. La **chiave primaria** è il campo che identifica in modo unico ogni riga. Spesso usiamo un campo **id** con numeri progressivi.

- **PRIMARY KEY** : il valore identifica una sola riga e non può mancare.
- **AUTO_INCREMENT** : MySQL assegna automaticamente il prossimo numero a un **id** intero.
- **NOT NULL** : il campo è obbligatorio.
- **UNIQUE** : un valore può comparire una sola volta, per esempio un'email.
- **DEFAULT** : assegna un valore quando non ne viene fornito uno.
- **CHECK** : accetta solo valori che rispettano una condizione, per esempio un prezzo non negativo.

```
prezzo DECIMAL(8, 2) NOT NULL CHECK (prezzo >= 0)
attivo BOOLEAN NOT NULL DEFAULT TRUE
email VARCHAR(100) NOT NULL UNIQUE
```

La chiave primaria non è necessariamente un dato significativo per l'utente. Nome e cognome possono ripetersi, mentre un **id** resta un identificatore semplice e stabile.

CAPITOLO 6

Creare una tabella con MySQL

L'istruzione **CREATE TABLE** trasforma il modello progettato nella struttura concreta del database. Il nome della tabella è seguito, tra parentesi, dall'elenco dei campi e delle loro regole.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(50) NOT NULL,  
  cognome VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  data_nascita DATE,  
  attivo BOOLEAN NOT NULL DEFAULT TRUE  
);
```

La tabella contiene soltanto una struttura: dopo aver eseguito il comando esiste **clienti**, ma non contiene ancora record. Per aggiungere o leggere dati servono istruzioni SQL dedicate.

CAPITOLO 7

Creare e rimuovere tabelle

NOTA

DROP TABLE elimina definitivamente una tabella e tutti i dati che contiene. Va usato solo quando si è certi del risultato.

CREATE TABLE IF NOT EXISTS

Se si esegue **CREATE TABLE** con un nome già presente, MySQL restituisce un errore. Aggiungendo **IF NOT EXISTS**, MySQL crea la tabella solo se non esiste ancora.

```
CREATE TABLE IF NOT EXISTS clienti (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nome VARCHAR(50) NOT NULL,  
  cognome VARCHAR(50) NOT NULL,  
  email VARCHAR(100) NOT NULL UNIQUE,  
  data_nascita DATE,  
  attivo BOOLEAN NOT NULL DEFAULT TRUE  
);
```

Se **clienti** esiste già, l'istruzione non ne modifica la struttura: MySQL salta semplicemente la creazione.

DROP TABLE

DROP TABLE rimuove una tabella dal database. Dopo l'esecuzione non restano né la struttura né i record della tabella.

```
DROP TABLE clienti;
```

La variante **IF EXISTS** evita un errore quando la tabella indicata non è presente; non impedisce la cancellazione quando la tabella esiste.

```
DROP TABLE IF EXISTS clienti;
```

CAPITOLO 8

Colonne di log

NOTA

Le colonne di log temporali registrano automaticamente quando un record viene creato e quando viene modificato.

Queste due colonne possono essere aggiunte alla definizione di una tabella:

```
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
updated_at DATETIME ON UPDATE CURRENT_TIMESTAMP DEFAULT NULL
```

- **created_at** riceve automaticamente data e ora correnti quando il record viene creato.
- **updated_at** parte da **NULL** e riceve automaticamente data e ora correnti alla prima modifica del record, poi a ogni modifica successiva.

Le colonne possono essere collocate insieme agli altri campi nella parentesi di **CREATE TABLE**.